

Cuisson d'une pièce thermo-formée

Projet MO102 - Matlab

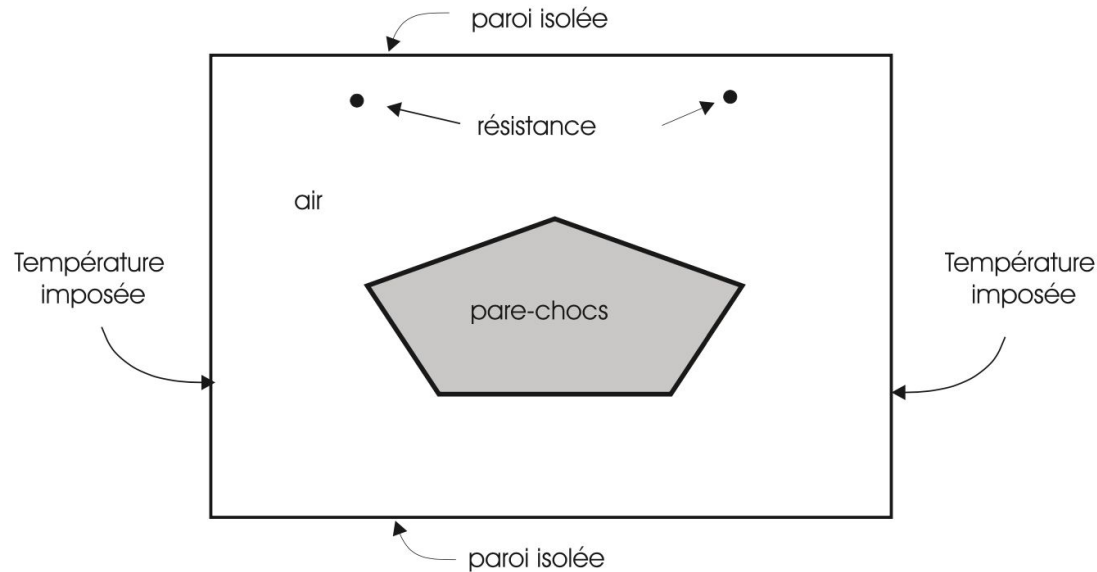
CHRÉTIENNE Thomas
CUEL Tom



Présentation du problème direct



Le problème direct



$$\begin{cases} -\operatorname{div}(\kappa \nabla T) = f & \text{dans } \Omega \\ T = T^D & \text{sur } \Gamma_D \\ \kappa \frac{\partial T}{\partial n} = 0 & \text{sur } \Gamma_I. \end{cases}$$

FIGURE 1 – Schéma du four

Réécriture du problème direct

$$\left\{ \begin{array}{ll} -\operatorname{div}(\kappa \nabla T) = f & \text{dans } \Omega \\ T = T^D & \text{sur } \Gamma_D \\ \kappa \frac{\partial T}{\partial n} = 0 & \text{sur } \Gamma_I. \end{array} \right.$$

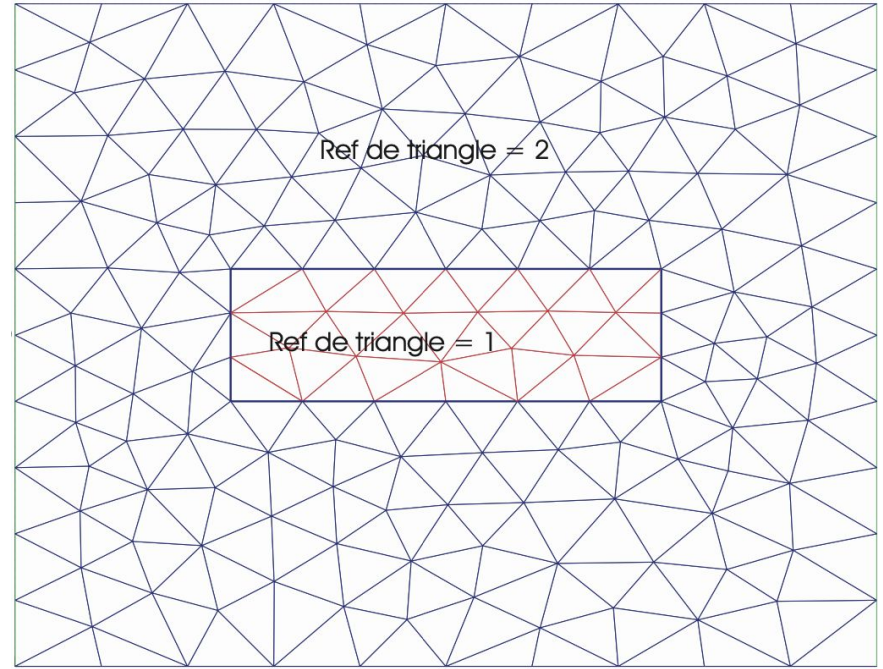


$$\left\{ \begin{array}{l} \text{Trouver } T \in \tilde{T}^D + V_0 \text{ tel que} \\ \int_{\Omega} \kappa \nabla T \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega, \quad \forall v \in V_0. \end{array} \right.$$

Représentation des données

À partir d'un fichier de maillage, on accède:

- au **nombre de points**
- au **nombre de triangles**
- aux **coordonnées des sommets**
- au **tableau de référence** indiquant pour chaque triangle un numéro de référence, permettant de repérer son appartenance aux différents domaines
- au **tableau des triangles** référencés avec les sommets correspondants
- au **numéro de référence** du triangle



Plan / Code

- Réécriture du problème avec la méthode de discrétisation des éléments finis
 - construction de K
 - prise en compte des résistances
 - construction du second membre
- Simplification de K et du second membre
- Résolution du problème direct
- Affichage graphique du maillage
- Obtention de la température maximale

```
function probleme_direct_presentation(Nbpt, Nbtri, Coorneu, Refneu, Numtri, Reftri, fichier)
%=====  
%Calcul de Kmat  
%=====  
Kmat=matrice_K(Nbpt, Nbtri, Coorneu, Numtri, Reftri);  
  
%=====  
%Prise en compte des résistances  
%=====  
n=input("donner le nombre de résistances : ");  
tableau_resistance=resistances(n);  
disp("tableau des résistances")  
disp(tableau_resistance)  
  
%=====  
%Calcul du second membre  
%=====  
f=f_part1(Nbpt, Nbtri, Coorneu, Numtri, tableau_resistance,n);  
  
%=====  
%Choix de la température de la pièce  
%=====  
TD=input("entrer la température de la pièce : ");  
td=vect_td(Nbpt, Refneu, TD);  
  
%=====  
%Modification du second membre  
%=====  
secondmembremod=second_membre_mod_part1(Nbpt, Refneu, f, Kmat, td, TD);  
  
%=====  
%Modification de K  
%=====  
Kmod=Kmodifie(Nbpt, Refneu, Kmat);  
  
%=====  
%Résolution de l'équation différentielle  
%=====  
T0=Kmod\secondmembremod;  
  
%=====  
%Affichage du maillage en couleur  
%=====  
affichemaillage(fichier,'titre',T0);  
  
%=====  
%Obtention de la température maximale dans le four  
%=====  
max = temperature_max(T0, Nbpt);  
disp("température max")  
disp(max)
```

Discrétisation par éléments finis (1)

Le principe de la méthode des éléments finis consiste à construire un sous-espace vectoriel de dimension finie N , noté V_0^N , de l'espace V_0 ainsi qu'une approximation $\tilde{T}^{DN}$ de $\tilde{T}^D$ et à chercher une solution T^N **approchée**, solution du problème variationnel **discret** :

$$\left\{ \begin{array}{l} \text{Trouver } T^N \in \tilde{T}^{DN} + V_0^N \text{ tel que} \\ \int_{\Omega} \kappa \nabla T^N \cdot \nabla v \, d\Omega = \int_{\Omega} f v \, d\Omega, \quad \forall v \in V_0^N. \end{array} \right. \quad (3)$$

Discrétisation par éléments finis (2)

On obtient :

$$\mathbb{K} \vec{t} = \vec{f} - \vec{g}.$$

$\mathbb{K}$ est une matrice d'ordre N donnée par :

$$\mathbb{K}_{ij} = \int_{\Omega} \kappa \nabla w_i \cdot \nabla w_j d\Omega, \quad 1 \leq i, j \leq N,$$

et $\vec{f}$ et $\vec{g}$ sont des vecteurs de $\mathbb{R}^N$ définis par :

$$f_i = \int_{\Omega} f w_i d\Omega \text{ et } g_i = \int_{\Omega} \kappa \nabla \tilde{T}^{DN} \cdot \nabla w_i d\Omega, \quad 1 \leq i \leq N.$$

Implémentation en Matlab

Construction de la matrice élémentaire locale

```

=====
%Construction de la matrice d'assemblage de K
=====

% la fonction prend comme arguments :
% - Coorneu(i) = coordonnées du sommet i
% - Numtri = tableau des triangles référencés
%           avec les sommets correspondants
% - l = numéro du triangle considéré
% - ka = coefficient dépendant du milieu dans lequel est le triangle

% tabx(i,j)=xi-xj
tabx=[0 x1-x2 x1-x3;x2-x1 0 x2-x3; x3-x1 x3-x2 0];
taby=[0 y1-y2 y1-y3;y2-y1 0 y2-y3; y3-y1 y3-y2 0];

% on suit la méthode de calcul de la partie théorique
delta = tabx(2,3)*taby(3,1) - tabx(3,1)*taby(2,3);
deltainv = 1/delta;
aire = abs(delta)/2;

grad = [taby(2, 3), -tabx(2, 3); taby(3, 1), -tabx(3, 1);
        taby(1, 2), -tabx(1, 2)];
grad = grad * deltainv;

mat = aire * ka * (grad * grad');

end

function [mat]=kloc_part1(Coorneu, Numtri, l, ka)

% obtention des coordonnées des sommets du triangle l
x1 = Coorneu(Numtri(l,1), 1);
x2 = Coorneu(Numtri(l,2), 1);
x3 = Coorneu(Numtri(l,3), 1);

y1 = Coorneu(Numtri(l,1), 2);
y2 = Coorneu(Numtri(l,2), 2);
y3 = Coorneu(Numtri(l,3), 2);
    
```

$$\mathbb{K}_{km}^{\ell} = \int_{T_{\ell}} \kappa \nabla w_i \cdot \nabla w_j d\Omega \text{ avec } i = g(\ell, k) \text{ et } j = g(\ell, m)$$

Savoir si un point est à l'intérieur d'un triangle

% la fonction prend comme arguments :

% - point = un point du plan de type (x,y)

% - sommetsTriangle = matrice 3x2 des coordonnées des sommets du triangle considéré

```
function estAInterieur = estPointDansTriangle(point, sommetsTriangle)
```

% coordonnées des sommets du triangle

A = sommetsTriangle(1,:);

B = sommetsTriangle(2,:);

C = sommetsTriangle(3,:);

% calcul des barycentres

alpha = ((B(2)-C(2))*(point(1)-C(1)) + (C(1)-B(1))*(point(2)-C(2))) / ((B(2)-C(2))*(A(1)-C(1)) + (C(1)-B(1))*(A(2)-C(2)));

beta = ((C(2)-A(2))*(point(1)-C(1)) + (A(1)-C(1))*(point(2)-C(2))) / ((B(2)-C(2))*(A(1)-C(1)) + (C(1)-B(1))*(A(2)-C(2)));

gamma = 1 - alpha - beta;

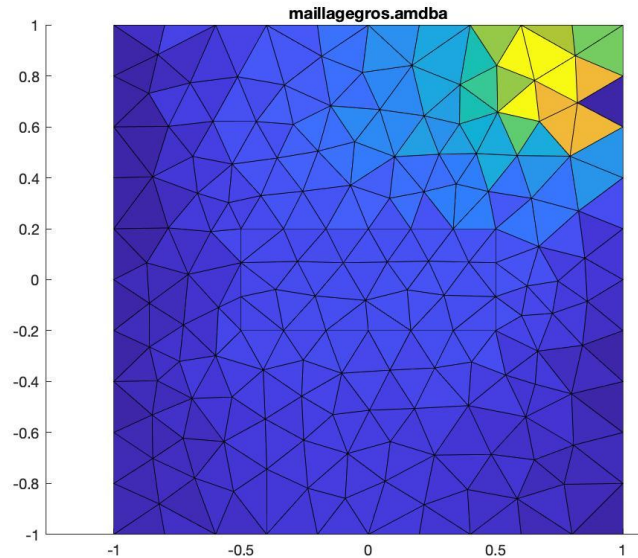
% vérification si les barycentres sont dans la plage [0, 1]

estAInterieur = alpha >= 0 && beta >= 0 && gamma >= 0 && alpha <= 1 && beta <= 1 && gamma <= 1;

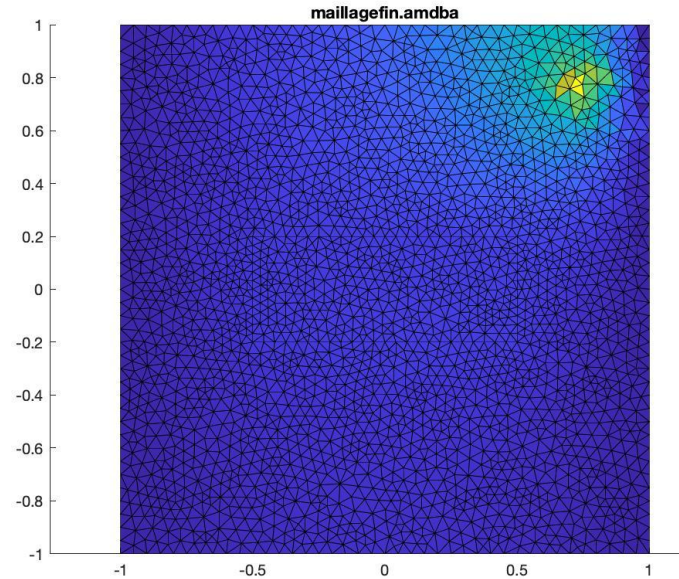
```
end
```

Résultats pour différents type de maillage

Maillage gros



Maillage fin



Cas : 1 résistance de 10 000 Ω placée en $(3/4, 3/4)$ dans un pièce à 300°K



Présentation du problème inverse



Notations et formulation du problème

$$T = T_0 + \sum_{r=1}^n \alpha_r \theta_r,$$

$A_r = \alpha_r A_0$ est la valeur de la résistance r

θ_r la température générée par une source unitaire

But : Minimiser :

$$J(\vec{\alpha}) = \frac{1}{2} \int_{\Omega_p} \left[\left(T_0 + \sum_{r=1,n} \alpha_r \theta_r \right) - T_{opt} \right]^2 d\Omega + \frac{\beta}{2} \|\vec{\alpha}\|_2^2$$

où β est un paramètre positif ou nul qui permet de prendre en compte l'importance relative des deux termes à minimiser.

Réécriture du problème

Minimiser :
$$J(\vec{\alpha}) = \frac{1}{2} \int_{\Omega_p} \left[\left(T_0 + \sum_{r=1,n} \alpha_r \theta_r \right) - T_{opt} \right]^2 d\Omega + \frac{\beta}{2} \|\vec{\alpha}\|_2^2$$



Minimiser :
$$J(\vec{\alpha}) = c - (\vec{\alpha} | \vec{b})_{\mathbb{R}^n} + \frac{1}{2} (\mathbb{A} \vec{\alpha} | \vec{\alpha})_{\mathbb{R}^n}.$$

Réécriture du problème (détail)

$$\begin{aligned} ((T_0 + \sum \alpha_k \vartheta_k) - T_{opt})^2 &= \left[(T_0 - T_{opt}) + \sum \alpha_k \vartheta_k \right]^2 \\ &= (T_0 - T_{opt})^2 + 2 \cdot (T_0 - T_{opt}) \cdot \sum \alpha_k \vartheta_k + \left(\sum \alpha_k \vartheta_k \right)^2. \end{aligned}$$

$$\begin{aligned} \text{d'où } J(\vec{\alpha}') &= \underbrace{\frac{1}{2} \int_{\Omega_p} (T_0 - T_{opt})^2 d\Omega}_{=c} + \underbrace{\int_{\Omega_p} (T_0 - T_{opt}) \cdot \sum \alpha_k \vartheta_k d\Omega}_{= \sum \alpha_k \int_{\Omega_p} (T_0 - T_{opt}) \vartheta_k d\Omega} + \underbrace{\frac{1}{2} \int_{\Omega_p} (\sum \alpha_k \vartheta_k)^2 d\Omega}_{= \frac{1}{2} \|\alpha\|^2} + \underbrace{\frac{\beta}{2} \|\alpha\|^2}_{= \frac{\beta}{2} \sum \alpha_k^2} \\ &= \sum \alpha_k \int_{\Omega_p} (T_0 - T_{opt}) \vartheta_k d\Omega = (\sum \alpha_k \vartheta_k) (\sum \alpha_k \vartheta_k) = \int_{\Omega_p} \sum \alpha_k^2 d\Omega \\ &= - \sum \alpha_k b_k \quad \text{Fubini} \left(= \sum_k \sum_j \alpha_k \alpha_j \vartheta_k \vartheta_j \right) \\ &= - (\alpha, b)_{\mathbb{R}^n}. \end{aligned}$$

$$A_{rr'} = \int_{\Omega_p} \vartheta_r \vartheta_{r'} d\Omega + \beta \delta_{rr'}.$$

$$b_r = \int_{\Omega_p} \vartheta_r (T_{opt} - T_0) d\Omega.$$

$$c = \frac{1}{2} \int_{\Omega_p} (T_0 - T_{opt})^2 d\Omega$$

$$\begin{aligned} \text{Or, } (A\vec{\alpha}', \vec{\alpha}')_{\mathbb{R}^n} &= \sum_{r=1}^n \sum_{r'=1}^n A_{rr'} \alpha_r' \cdot \alpha_{r'} \\ &= \sum_{r=1}^n \sum_{r'=1}^n \left(\int_{\Omega_p} \vartheta_r \vartheta_{r'} d\Omega + \beta \delta_{rr'} \right) \alpha_r' \cdot \alpha_{r'} \\ &= \int_{\Omega_p} \left(\sum_r \sum_{r'} \vartheta_r \vartheta_{r'} \alpha_r' \alpha_{r'} \right) d\Omega + \underbrace{\sum_r \sum_{r'} \beta \delta_{rr'} \alpha_r' \cdot \alpha_{r'}}_{= \beta \cdot \sum_r \alpha_r'^2} \end{aligned}$$

finalement :

$$J(\vec{\alpha}') = c - (\vec{\alpha}', \vec{b}) + \frac{1}{2} (A\vec{\alpha}', \vec{\alpha}')$$

Résolution du problème

A partir de là, on peut établir, cf. l'exercice suivant, que la fonctionnelle J admet un **point de minimum unique** $\vec{\alpha}_m$, solution du système linéaire

$$\mathbb{A}\vec{\alpha}_m = \vec{b}.$$

On résout numériquement les $n+1$ problèmes directs qui permettent de calculer les températures T_0 et θ_r

On obtient les solutions :

$$\begin{cases} T_0^N = \tilde{T}^{DN} + \sum_{j=1,N} t_{j,0} w_j. \\ \theta_r^N = \sum_{j=1,N} t_{j,r} w_j \end{cases} \quad \text{pour } 1 \leq r \leq n.$$

Construction de la matrice A (1)

```
%=====
%Construction de la matrice A
%=====
```

```
% la fonction prend comme arguments :
```

- % - Nbtri = nombre de triangle
- % - Coorneu(i) = coordonnées du sommet i
- % - Numtri = tableau des triangles référencés avec les sommets correspondants
- % - Reftri(i) = numéro de référence du triangle i
- % - beta = nombre permettant de contrôler l'importance des termes à minimiser
- % - matrice_temperatures = matrice des températures pour chaque résistance, toujours de valeurs A0
- % - nb_resistances = nombre de résistances

Construction de la matrice A (2)

```
function [A]=matrice_A(Nbtri, Coorneu, Numtri, Reftri, beta, matrice_temperatures, nb_resistances)

% la matrice des beta*delta
A=beta*eye(nb_resistances);

% chaque triangle dans omega_p va ajouter des valeurs à A liées aux températures trouvées précédemment
for l = 1 : Nbtri
    if Reftri(l)==1          % on regarde si on est dans le pare-choc

        % matrice d'assemblage de A
        M=kloccalc_part2_A(Coorneu, Numtri, l);    %Matrice locale

        % assemblage de A
        for r = 1 : nb_resistances
            for s = 1 : nb_resistances
                for i = 1 : 3
                    for j = 1 : 3
                        A(r,s) = A(r,s) + M(i,j)*matrice_temperatures(Numtri(l,i),r)*matrice_temperatures(Numtri(l,j),s);
                    end
                end
            end
        end
    end
end
end
```

Architecture du code complet (1)

```
% la fonction prend comme arguments :  
% - Nbpt = nombre de sommets  
% - Nbtri = nombre de triangle  
% - Coorneu(i) = coordonnées du sommet i  
% - Refneu(i) = numéro de référence du sommet i  
% - Numtri = tableau des triangles référencés avec les sommets correspondants  
% - Reftri(i) = numéro de référence du triangle i  
% - fichier = nom du maillage du four que l'on étudie
```

```
function probleme_inverse_presentation(Nbpt, Nbtri, Coorneu, Refneu, Numtri, Reftri, fichier)
```

Architecture du code complet (2)

```
%=====
%Demande des données à l'utilisateur
%=====
beta=input("donner le nombre  $\beta$  : ");
Topt=input("entrer la température du pare-chocs souhaitée : ");
TD=input("entrer la température des bords du four : ");
    T0=zeros(Nbpt,1);
    for i=1:Nbpt
        T0(i)=TD;
    end
A0=input("résistance de base : ");

%=====
%Création du tableau de l'emplacement des résistances
%=====
nb_resistances=input("nombres de résistances souhaitées : ");
resistances=placement_des_resistances(Nbtri, Coorneu, Numtri, Reftri, nb_resistances);

%=====
%Construction de la matrice des températures
%=====
% On trouve ici les différents  $\theta_r$  que l'on calcule avec la résistance A0
% et qu'il faudra coefficenter avec les alpha_r plus tard dans le calcul
matrice_temperatures=matrice_temperatures_Tr(Nbpt, Nbtri, Coorneu, Refneu, Numtri, Reftri, resistances, nb_resistances, A0, TD);
```

Architecture du code complet (3)

```
%=====
%Construction de la matrice A
%=====|
A=matrice_A(Nbtri, Coorneu, Numtri, Reftri, beta, matrice_temperatures, nb_resistances);

%=====
%Création du vecteur différence entre la température voulue
%et celle des parois du four (Topt-TD)
%=====
Tb=Topt-TD;

%=====
%Construction du vecteur b
%=====
b=vect_b(Nbtri, Coorneu, Numtri, Reftri, Tb, matrice_temperatures, nb_resistances);

%=====
%Résolution du système
%=====
alpha_m=A\b;
```

Architecture du code complet (4)

```
%=====
%Calcul de la température en prenant en compte la pondération des
%résistances
%=====
T=T0;
for i=1:nb_resistances
    T=T+alpha_m(i)*matrice_temperatures(:,1);
end

%=====
%Affichage du maillage en couleur
%=====
affichagemaillage(fichier,'titre',T);

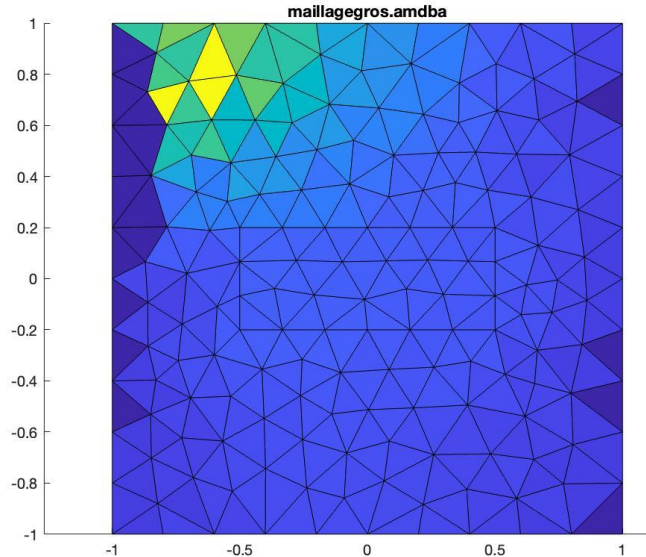
%=====
%Calcul de la norme L2 de alpha_m
%=====
norm_alpha_m=norm(alpha_m,2);
disp("norme sur L2 de alpha_m")
disp(norm_alpha_m)

%=====
%Calcul de l'écart de température en norme L2
%=====
ecart_norm_L2_de_temperatures=ecart_temperatures_norm_L2(Nbtri, Coorneu, Numtri, Reftri, T, Topt);
disp("norme L2 d'écart de température avec Topt")
disp(ecart_norm_L2_de_temperatures)
```

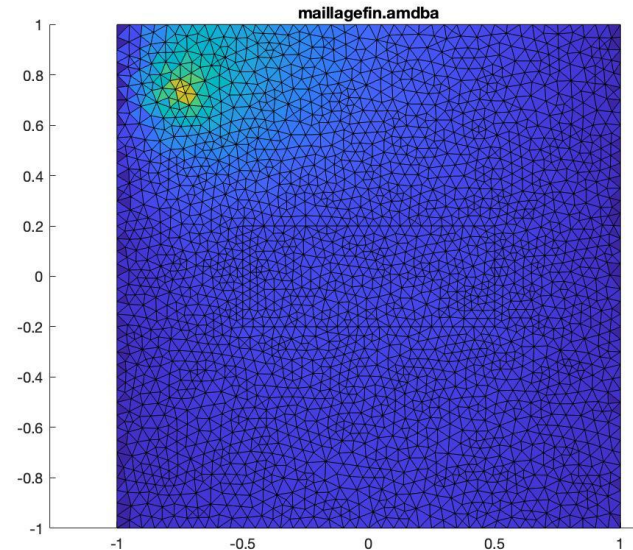
end

Résultats pour différents type de maillage

Maillage gros



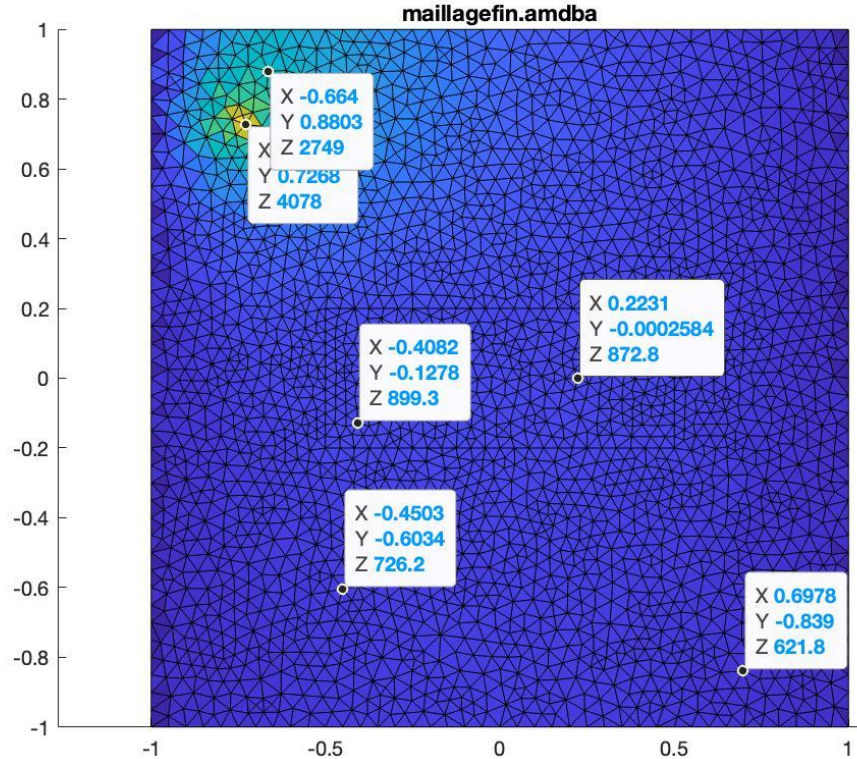
Maillage fin



Cas : $T(\text{bord_four}) = 300\text{K}$; $T_{\text{opt}} = 500\text{K}$; 6 résistances de $10\,000\,\Omega$ en $(\frac{3}{4}, \frac{3}{4})$, $(-\frac{3}{4}, \frac{3}{4})$, $(\frac{3}{4}, -\frac{3}{4})$, $(-\frac{3}{4}, -\frac{3}{4})$, $(\frac{3}{4}, 0)$, $(-\frac{3}{4}, 0)$. $\beta = 0.001$

Résultats pour différents type de maillage

Maillage fin





Optimisation globale du four



Notations et formulation du problème

β est fixée à 10^{-4} .

$$K(x_1, y_1, \dots, x_n, y_n) = J(\vec{\alpha}_m)$$

But : Évaluer numériquement :

$$\min_{(x_1, \dots, y_n)} K(x_1, \dots, y_n)$$

Réalisation :

Concevoir une fonction qui explore automatiquement un certain nombre de configuration et qui détermine, parmi celles-ci, la meilleure

AFFICHAGE RÉSULTATS - Maillage gros (1000 configurations testées)

Input :

nombre de triangles :
304

nombre de sommets de triangle / points :
173

1 : Problème direct
2 : Problème inverse
3 : Optimisation globale du four
choisir la partie que l'on veut traiter : 3

beta =
1.0000e-04

entrer la température du pare-chocs souhaitée : 500
entrer la température des bords du four : 300
résistance de base : 10000
résistance de base :
10000

nombre de configuration que vous voulez tester : 1000
nombre de configurations à tester
1000

nombres de résistances souhaitées : 6
nombres de résistances souhaitées :
6

Output :

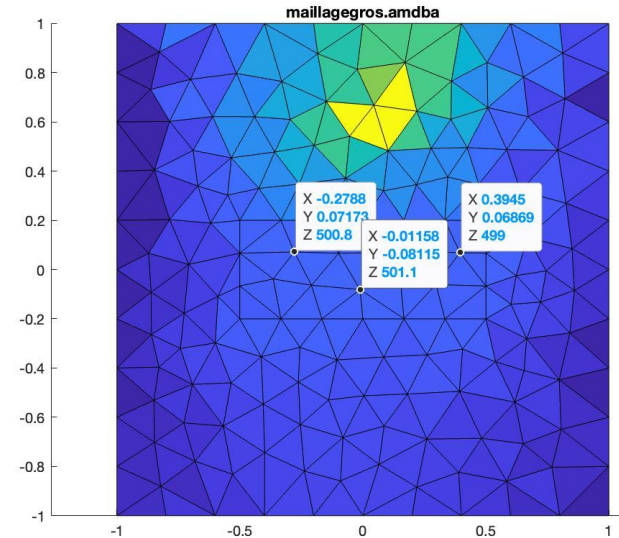
meilleure répartition des résistances
0.0446 -0.5000 0.7920 0.5000 0.2606 -0.3333
0.6679 -0.0667 0.4847 -0.0667 -0.3463 -0.2000

meilleur alpha, pondération des résistances
0.0067
0.0056
0.0071
0.0023
0.0023
0.0008

norme associée
22.1259

nombre de configurations testées
1000

Représentation :





Merci de votre écoute

CHRÉTIENNE Thomas

CUEL Tom



Annexe : Algorithme utile : boucle sur les configurations à tester (1)

```
% on boucle sur les configurations à tester
k=nb_config_testees;
while k>0

%=====
%Utilisation de la fonction random :
%=====
    % donne 6 nombres entre 1 et Nbpt
    % refait si le nombre a déjà été tiré
    % fait attention à ce que la résistance ne soit pas sur le pare-choc
    resistances=recup_6_resistances_aleatoire(Nbpt, Nbtri, Coorneu, Numtri, Reftri, nb_resistances);

%=====
%Recherche des différents  $\theta_r$  que l'on calcule avec la résistance  $A_0$ 
%=====
% il faudra ensuite les coefficienter avec les  $\alpha_r$ 
matrice_temperatures=matrice_temperatures_Tr(Nbpt, Nbtri, Coorneu, Refneu, Numtri, Reftri, resistances, nb_resistances,  $A_0$ , TD);

%=====
%Construction de la matrice A
%=====
A=matrice_A(Nbtri, Coorneu, Numtri, Reftri, beta, matrice_temperatures, nb_resistances);
```

Annexe : Algorithme utile : boucle sur les configurations à tester (2)

```
%=====
%Création du vecteur différence entre la température voulue et celle des parois du four
%=====
Tb=Topt-TD;

%=====
%Construction du vecteur b
%=====
b=vect_b(Nbtri, Coorneu, Numtri, Reftri, Tb, matrice_temperatures, nb_resistances);

%=====
%Résolution du système
%=====
alpha_m=A\b;
```

Problème dans la partie 2 :

vecteur des coefficients de pondération des résistances

-0.0540
0.0046
-0.0142
-3.8644
0.5273
4.1557

Annexe : Algorithme utile : boucle sur les configurations à tester (3)

```
%=====
%Vérification de la positivité des résistances
%=====
% on incrémente k que lorsqu'on a visité une configuration valable
if resistance_positive(alpha_m, nb_resistances)==1
    %=====
    %Calcul de la température en prenant en compte la pondération des résistances
    %=====
    T=T0;
    for i=1:nb_resistances
        T=T+alpha_m(i)*matrice_temperatures(:,1);
    end

    %=====
    %Calcul de la norme2 de T-Topt
    %=====
    norm2_temp=ecart_temperatures_norm_L2(Nbtri, Coorneu, Numtri, Reftri, T, Topt);

    %=====
    %Comparaison de cette configuration avec celle qui minimise le plus la norme
    %=====
    if norm2_temp < meilleure_norme
        meilleure_temperatures=T;
        meilleure_norme=norm2_temp;
        meilleure_tableau_resistances=resistances;
        meilleur_alpha=alpha_m;
    end

    k=k-1;
end
```


Annexe : code de la fonction locale (problème inverse (1))

```
%=====
%Construction de la matrice d'assemblage de la matrice A
%=====

% la fonction prend comme arguments :
% - Coorneu(i) = coordonnées du sommet i
% - Numtri = tableau des triangles référencés avec les sommets correspondants
% - l = numéro du triangle considéré
```

```
function [M]=kloccalc_part2_A(Coorneu, Numtri, l)

% obtention des coordonnées des sommets du triangle l
x1 = Coorneu(Numtri(l,1), 1);
x2 = Coorneu(Numtri(l,2), 1);
x3 = Coorneu(Numtri(l,3), 1);

y1 = Coorneu(Numtri(l,1), 2);
y2 = Coorneu(Numtri(l,2), 2);
y3 = Coorneu(Numtri(l,3), 2);
```

Annexe : code de la fonction locale (problème inverse (2))

```
% tabx(i,j)=xi-xj
tabx=[0 x1-x2 x1-x3;x2-x1 0 x2-x3; x3-x1 x3-x2 0];
taby=[0 y1-y2 y1-y3;y2-y1 0 y2-y3; y3-y1 y3-y2 0];

delta = tabx(2,3)*taby(3,1) - tabx(3,1)*taby(2,3);
aire = abs(delta)/2;
```

```
% en utilisant la formule de l'annexe
M=zeros(3,3);
for i=1:3
    for j=1:3

        if i==j % si on a un lambda au carré
            M(i,j)=aire*2*2/(4*3*2);
        else % sinon
            M(i,j)=aire*2/(4*3*2);
        end

    end
end
end
```